Land, not tokens

A design for land qualities, agriculture, mining, fisheries and industry in Rise of Mankind, replacing the point resources of the map

What this document proposes. Resources stop being objects on single plots. Every plot, region and sea gets measurable qualities: temperature, water, growing season, soil, ore grade and depth, and fish. A city produces goods (wheat, coffee, iron, steel, arms) from the land it works, the knowledge it holds and the buildings it has, in quantities that are made, used up, stored and traded. The resource types of today survive only as the names of goods, which lets the engine's existing paths (building and unit requirements, happiness and health, the trade screen) keep working.

How it was made. The facts about today's game come from the mod's XML records and DLL source at GitLab `main 3bec049`, read by `research/economy/tools/extract.py`. The maps and simulations use the WorldBuilder map *Earth 18 preset civs, Huge* (124 by 68 plots, latitude -90 to 90 degrees) and are produced by `tools/figures.py`; every number in the text is read from what that script computed. Parameters that are design choices rather than facts are marked as assumptions and collected in section 15.

Contents

1. The game today
2. The proposal in brief
3. The land: climate, soil and regions
4. Agriculture
5. Seeds and the spread of crops
6. Minerals: provinces, grades and depth
7. The sea
8. Goods and quantities
9. Industry and war
10. Trade and prices
11. How the mechanics interlock
12. Fitting it into the engine
13. Three ways to build it
14. Roadmap and how each step is judged
15. Assumptions and open questions

Appendix: parameters and notation

1. The game today

This section describes what the game does now, from its records and code, before any proposal.

1.1 Resources are tokens on plots

- The mod defines **73 resource types** (`CIV4BonusInfos.xml`). 52 of them can stand on a map plot; the others are produced by buildings or come later in the game.
- A plot holds **at most one** resource: `CvPlot` stores it in a single field, `m_eBonusType`.
- The Earth map places **692** resource plots on 3077 land plots and 5355 water plots.
- A resource counts for a trade network once it is connected. A second copy adds nothing to the network that holds it; it can only be traded away. Happiness and health from a luxury are +1 or +2 for every city, whatever the city's size.

1.2 Latitude barely shapes where things are

Each resource has a latitude window (`iMinLatitude`, `iMaxLatitude`) used when a map script places resources at random. Of the 52 natural resources, **33 have the window 0 to 80 degrees**, which covers almost all land. Hand-made WorldBuilder maps ignore the windows altogether. The game's latitude function, `CvPlot::getLatitude`, returns the absolute value; only `getLatitudeMinutes` keeps the hemisphere's sign, and nothing in the economy uses it.

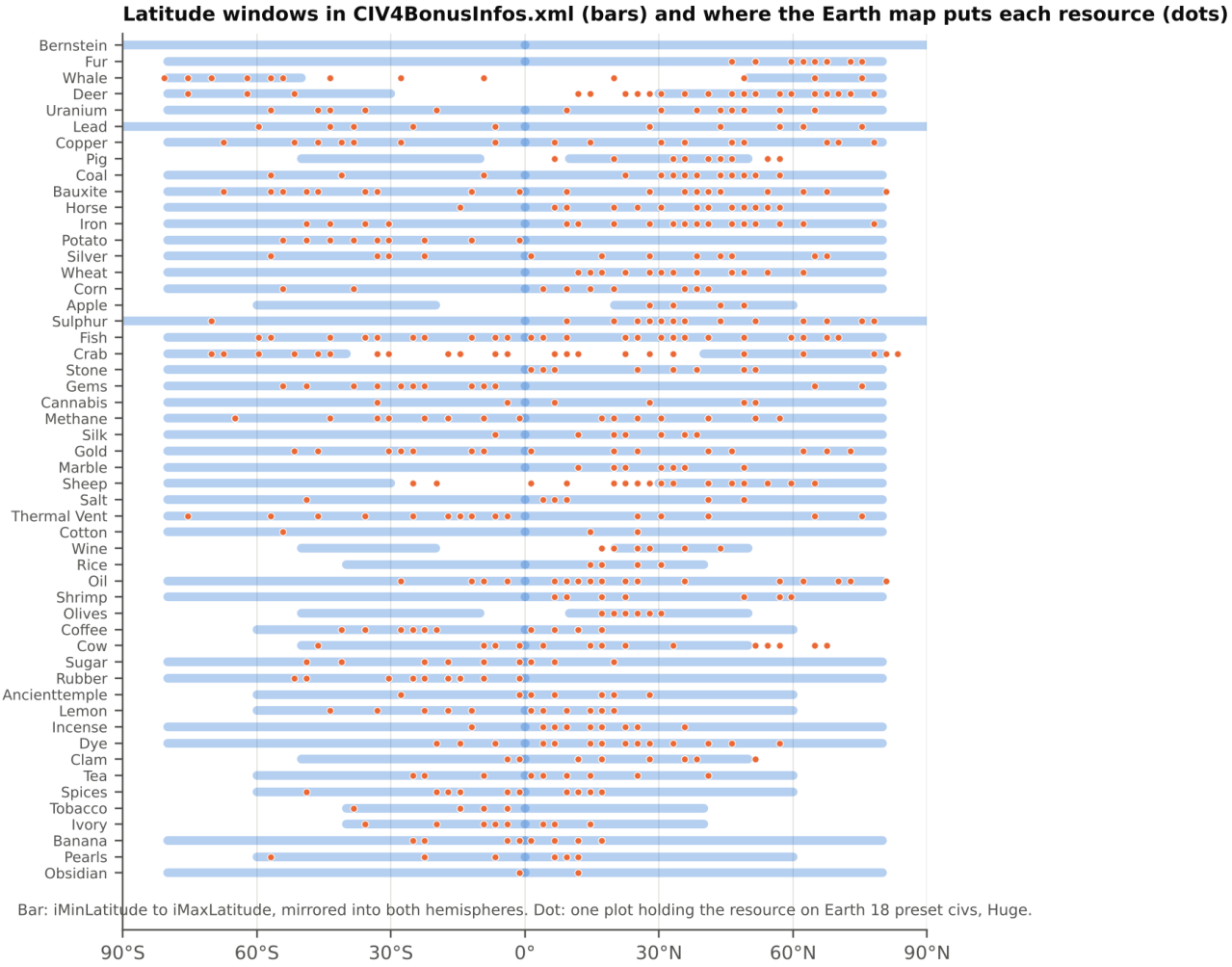


Figure 1. For each natural resource, its latitude window from the XML (blue bars, shown in both hemispheres) and the latitude of every plot the Earth map gives it (orange dots). Sorted by the median distance of the dots from the equator; resources the map does not place are at the top. Most windows span the whole habitable range; where a resource sits is decided by the map maker or the terrain and feature it needs.

1.3 Factories are gates, not flows

23 buildings produce a resource (`FreeBonus`); 14 of them need other resources first. The requirement is only that the inputs be present in the city's network: **nothing is used up**. A Steel Mill needs Coal and Iron and gives the network one Steel; a Weapon Factory needs Sulphur and one of Steel, Iron or Copper and gives Ammo. Where the XML says `-1`, the number of copies comes from the world size (`CvGame::getNumFreeBonuses`), 7 on a Huge map.

Building	Gives	Copies	Needs	And one of
Glasssmith	Glass	1	Stone	Salt
Steel Mill	Steel	1	Coal	Iron
Replicator Plant	Personal Replicators	world size	Chemicals	-
Nanite Lab	Nanobots	world size	Nanotubes	-
Weapon Factory	Ammo	world size	Sulphur	Steel, Iron, Copper
Rubber Factory	Vulcanized Rubber	world size	Rubber	Sulphur, Lead
Aluminum Factory	Aluminum	1	Bauxite	-
Oil Refinery	Diesel	1	Oil	-
Chemical Plant	Chemicals	world size	Sulphur	Diesel, Oil
Rubber Plant	Synthetic Rubber	world size	Chemicals	-
Fertilizer Plant	Fertilizers	world size	-	Chemicals
Biofuel Refinery	Biofuel	world size	-	Sugar, Corn, Potato
Brewery	Alcohol	world size	-	Apple, Wheat, Rice, Sugar, Corn
Nano Research Department	Nanotubes	world size	Coal	-

1.4 Mines already have depths; fields and seas do not

RoM has three mine improvements, each built by workers once a technology is known: Mine (Mining), Shaft Mine (Explosives) and Modern Mine (Seismology). Their yields are food, production and commerce [0, 1, 0], [0, 2, 1] and [-1, 8, 1]; a Mine also turns into a Shaft Mine by itself after 40 turns of work (`iUpgradeTime`). The same upgrade mechanism grows the Latifundium through five stages and the Cottage into a Town. Farms and fishing boats have one level each, and the sea's food sits on 94 resource plots of the Earth map.

2. The proposal in brief

2.1 Principles

- **The land is the resource.** Every land plot has a climate (temperature, water, growing season, hemisphere) and a soil; every plot lies in geological provinces with an ore grade and depth for each mineral; every sea plot has a carrying capacity for fish.
- **Production is land times work times knowledge.** A farm grows the crop its owner chooses among those the civilization knows and the land suits; a mine extracts what its province holds, down to the depth its technology reaches; a fishing fleet takes what the sea renews.
- **Goods are quantities.** They are made, moved within a trade network, used up by buildings, units and citizens, stored and traded.
- **No tokens on the map.** No plot holds a resource. What the player sees is the land itself, through overlays, the plot help text and the improvements built on it.
- **Resource types become goods.** The entries of `CIV4BonusInfos.xml` stay, as the list of goods. A city that makes or imports enough of a good holds it as a resource count, so every rule that reads resources today keeps working.

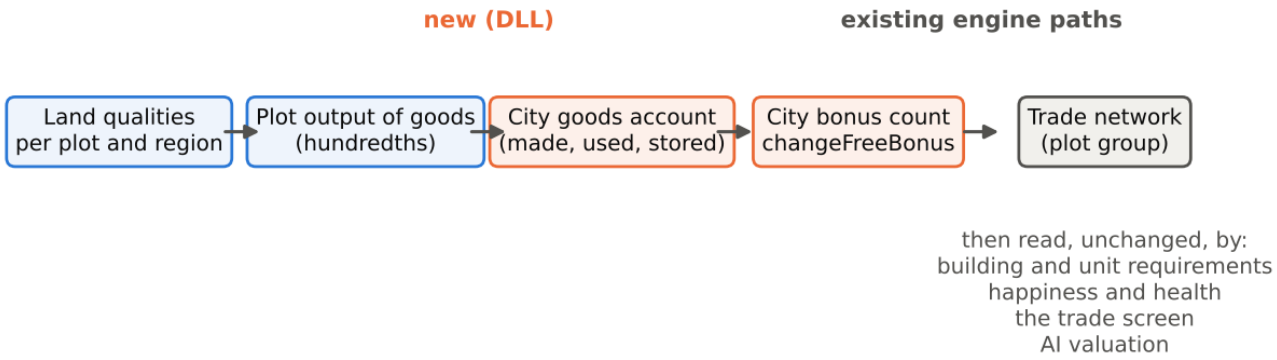


Figure 2. How goods made from land reach the rest of the game. The two blue and two orange steps are new DLL code; the city's resource count is fed through `CvCity::changeFreeBonus`, which already updates the city's trade network (`updatePlotGroupBonus`). Everything to the right reads those counts as it does now.

2.2 Today and proposed

	Today	Proposed
Where goods come from	A resource token on one plot, improved by the matching improvement	Land qualities of the worked plot and its region
Climate and hemisphere	Implied by terrain and features; latitude windows are loose and used only by random maps	Modelled per plot: temperature, water, growing season, warmest month
Crops	Fixed where the map put them	Planted wherever the land suits and the civilization knows the crop
Minerals	One token; one to three improvement levels	Provinces with grade, layered reserves, depth by technology, exhaustion
Sea	Fish, crab, clam and whale tokens	Carrying capacity of every sea plot; stocks that regrow and can collapse
Quantity	Present or absent; extra copies only for trade	Made, used, stored, traded per turn
Factories	Need inputs present; use nothing	Turn inputs into outputs at a rate, limited by the scarcest input
Luxuries and health	+1 or +2 per city whatever its size	Grow with consumption per head, with diminishing returns
Weapons	Gate a unit if the resource is present	Units cost goods to build, and stores of arms run down in war

3. The land: climate, soil and regions

A plot's qualities are computed once when the game starts, from the map, and change only through play (soil wear, depletion, improvements). The model below uses only what every map already has: each plot's row (its latitude), terrain, plot type (flat, hills, peak), feature, river, and its distance to water.

3.1 Temperature and seasons

For a plot in row y of a map H rows high, between latitudes ϕ_{bottom} and ϕ_{top}:

$$\varphi(y) = \varphi_{bottom} + \left(y + \tfrac{1}{2}\right)\frac{\varphi_{top} - \varphi_{bottom}}{H} \tag{1}$$

The annual mean temperature falls with the square of latitude and with altitude (hills 3 °C, peaks 12 °C cooler). The coefficients fit the zonal means of Earth: 27 °C at the equator, about 13 °C at 45° and -25 °C at the poles (assumption).

$$\bar{T} = 27 - 52\left(\frac{|\varphi|}{90}\right)^2 - \Delta_{elev} \tag{2}$$

Seasons have an amplitude that grows with latitude and with distance d from the sea (continentality), and they peak in July in the north and January in the south. This is where the hemisphere enters.

$$A = (1 + 0.28\,|\varphi|)\left(1 + 0.6\left(1 - e^{-d/5}\right)\right), \qquad T_m = \bar{T} + A\cos\frac{2\pi(m - m_0)}{12} \tag{3}$$

with month m from 1 to 12 and m_0 = 7 (July) where $\phi \geq 0$ and 1 (January) where $\phi < 0$.

$$L = \#\{\, m : T_m \geq 5\,^{\circ}\mathrm{C} \,\}, \qquad T_{min} = \min_m T_m \tag{4}$$

L is the growing season in months and T_{min} the coldest month, which decides frost-sensitive crops such as coffee and sugar.

3.2 Water

Rain from geography alone follows the global belts (wet equator, dry subtropics, wet mid-latitudes) and decays inland:

$$P_{geo} = \left(2200\,e^{-(\varphi/12)^2} + 1000\,e^{-((|\varphi| - 50)/14)^2} + 200\right)\left(0.3 + 0.7\,e^{-d/7}\right)\ \mathrm{mm} \tag{5}$$

Hand-made maps do not place climates where geography alone predicts. Figure 4 shows it on the Earth map: from geography alone, the plots the map maker drew as desert get a median of **844 mm**, more than grassland's **605 mm**, because this map puts its great deserts close to the equator. The terrain the map maker drew is therefore treated as an observation of climate and blended in with weight 0.7 (desert 150 mm, plains 650, grassland 1100, marsh 1600, tundra 400, snow 250, jungle 2200; assumptions), and a river adds irrigation water:

$$P = 0.3\,P_{geo} + 0.7\,P_{terrain}, \qquad W = P + 700 \cdot [\mathrm{river}] \tag{6}$$

After the blend the order is right: desert 358 mm, plains 647, grassland 1018. Temperature needs no blend: geography alone already ranks the drawn terrains as a map maker would (snow -18 °C, tundra -7, grassland 14, plains 18, desert 19).

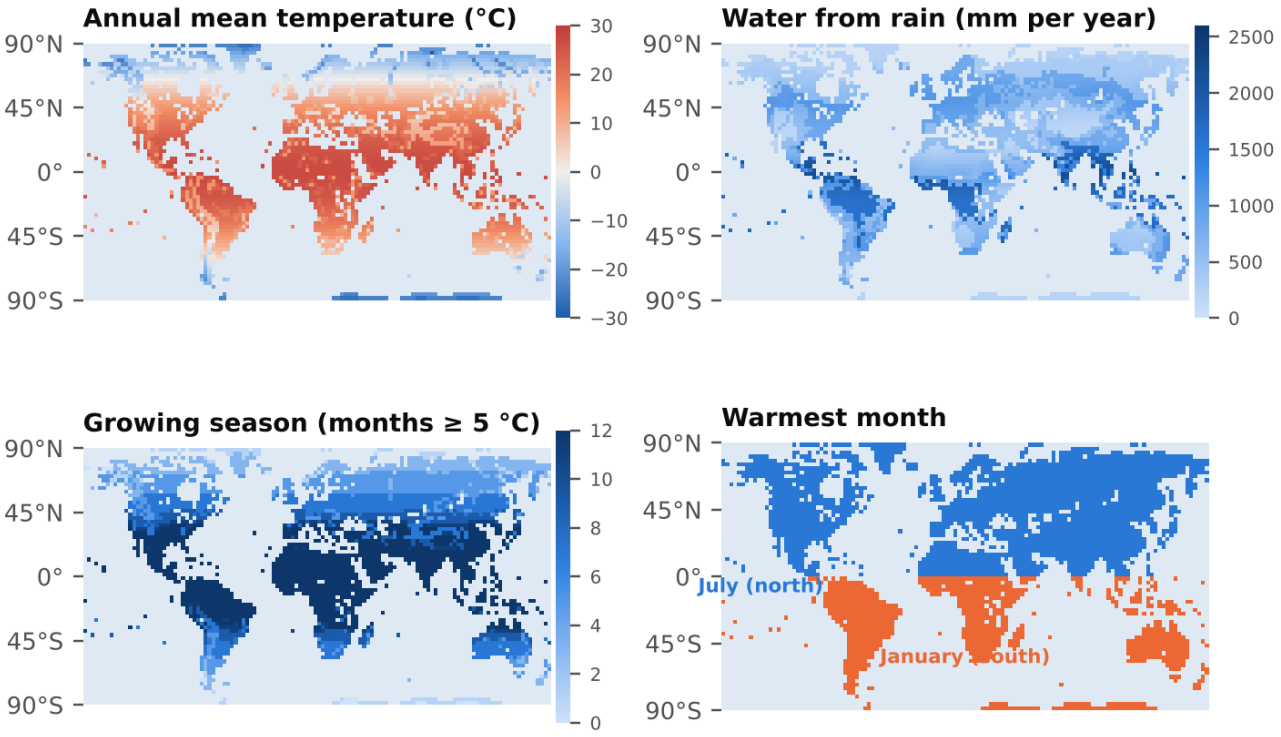


Figure 3. The climate model on the Earth map: annual mean temperature, water from rain (blended with the drawn terrain), growing season, and the warmest month, which flips at the equator.

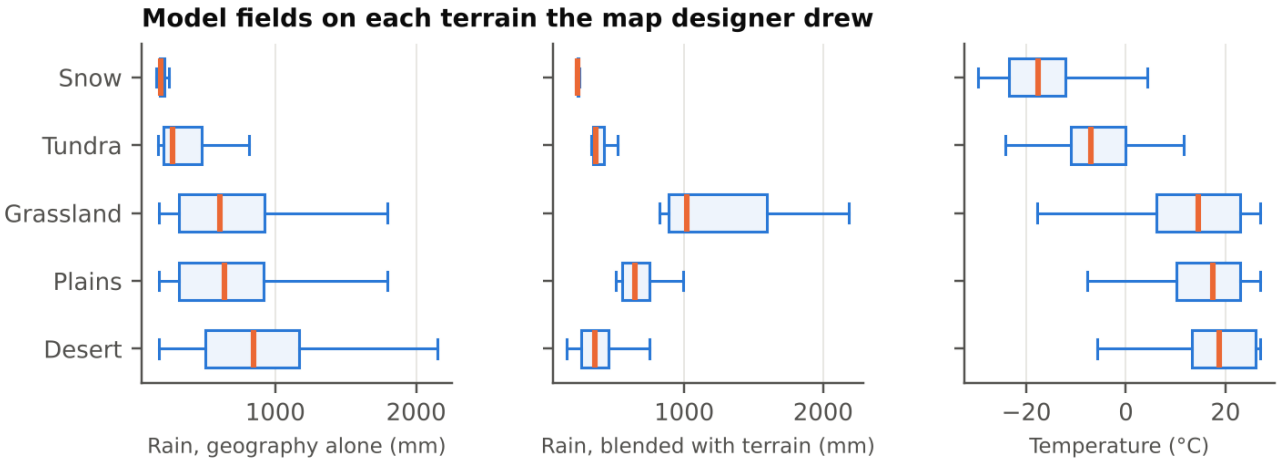


Figure 4. The model fields on the plots of each terrain the map maker drew (box: middle half; line: median; whiskers: range without outliers). Geography alone gets the deserts wrong on this map; blending the drawn terrain in puts them right. Plots: desert 518, plains 782, grassland 1057, tundra 403, snow 317.

3.3 Soil

Soil fertility *S* in [0, 1] starts from the terrain (grassland 0.8, plains 0.65, marsh 0.5, tundra 0.3, desert 0.15; assumptions), +0.15 beside a river (flood plains), -0.1 on hills, 0 on peaks. It then changes with use: intensive farming wears it, fallow and manure restore it, and fertilisers (already a manufactured resource) add to it:

$$S_{t+1} = S_t - \delta\, u_t + \rho\,(S_0 - S_t) + \phi\, f_t, \qquad 0 \le S \le 1 \tag{7}$$

with *u* the intensity of use (1 for a farm worked every turn), δ the wear per turn, ρ the recovery rate and *f* the fertiliser applied.

3.4 Regions

The user's idea of qualities that belong to a region or a landmass can be built three ways. They differ in what a "region" is.

Option	What a region is	For	Against
A. Landmass	The engine's `CvArea`: one per continent or island, already computed	No new map data; easy to explain ("Africa grows coffee")	Too coarse: a continent spans many climates
B. Climate class per plot	A class from the model fields (figure 5); contiguous plots of one class form a region	Follows latitude, hemisphere, altitude and coast; automatic on every map	Regions have no names or history
C. Named regions	Clusters of plots of one class on one landmass, named at map start (e.g. "the Nile valley"), with region-wide attributes such as a famous crop or ore	Gives places identity; regions can carry events and trade reputations	Needs a clustering step and naming data

Recommended: qualities per plot (option B underneath), shown and reasoned about by named region (option C) in the interface and in AI plans. Shares of the Earth map's land by class: tropical 31 %, arid 9 %, temperate 22 %, continental and boreal 13 %, polar 24 %.

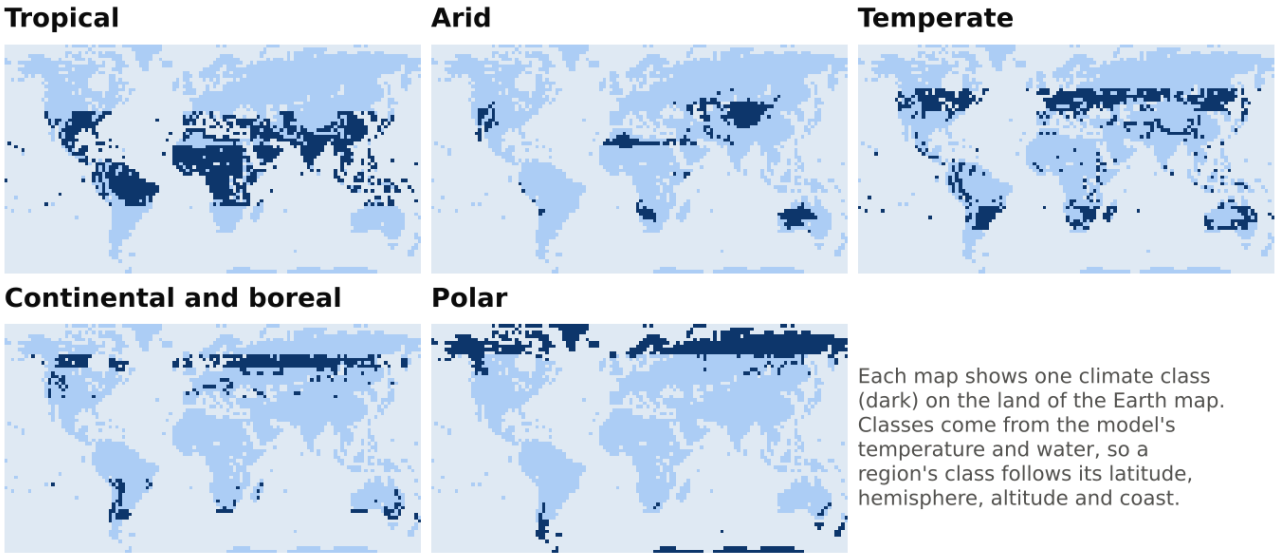


Figure 5. The five climate classes of the model on the Earth map, one map per class. Thresholds: tropical: mean 18 °C or more and not arid; arid: less than 400 mm of water; temperate 8 to 18 °C; continental and boreal -2 to 8 °C; polar below -2 °C.

4. Agriculture

4.1 How well a crop grows on a plot

Each crop *c* has a response to temperature and to water: zero outside a window, rising to full, a plateau, then falling. With the four corners $a < b \le c < d$ of the window:

$$g(x; a, b, c, d) = \max\left(0,\ \min\left(1,\ \frac{x-a}{b-a},\ \frac{d-x}{d-c}\right)\right) \tag{8}$$

The growing season and frost act the same way: a crop needing L_c months gets g_L = clamp((*L* - L_c + 2)/2), and one that cannot stand a coldest month below F_c gets g_F = clamp((T_{min} - F_c + 3)/3). The suitability of the plot multiplies the climate terms and weights the soil with an exponent β = 0.5 (assumption), so poor soil hurts but does not forbid:

$$s_c = g_T(\bar{T})\, g_W(W)\, g_L(L)\, g_F(T_{min})\, S^{\beta} \tag{9}$$

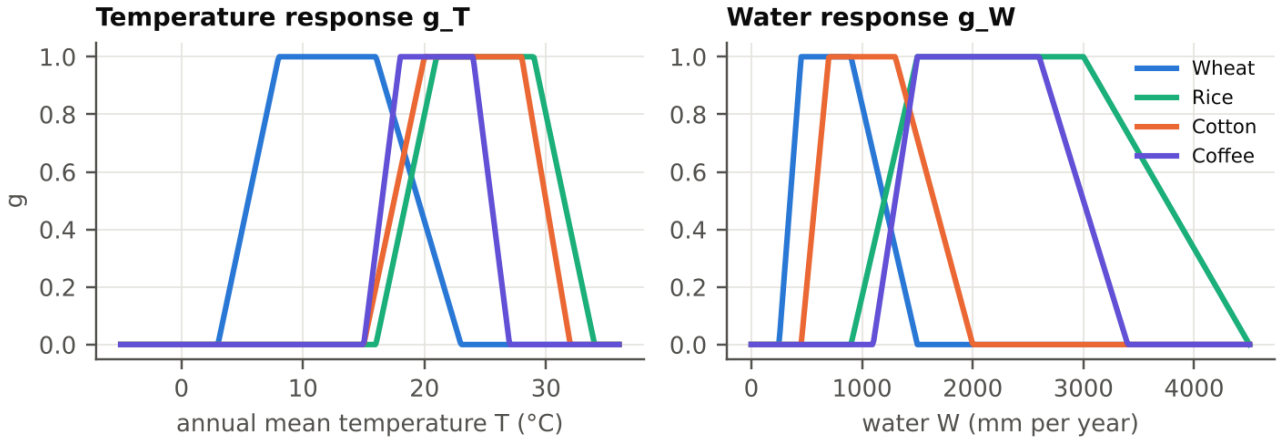


Figure 6. Temperature and water responses of four crops. The windows are approximate agronomic ranges (assumptions; appendix table A1) and would be XML fields of each crop, so modders tune them without code.

Product or law of the minimum. Liebig's law of the minimum takes the worst factor instead of the product: *s* = min(g_T, g_W, …). The product penalises a plot that is merely adequate on several counts; the minimum forgives it. For coffee, the share of the temperature and water grid of figure 7 with $s \ge 0.5$ is 19 % under the product and 20 % under the minimum. The product is recommended: it rewards the best land more, which gives regions their character.

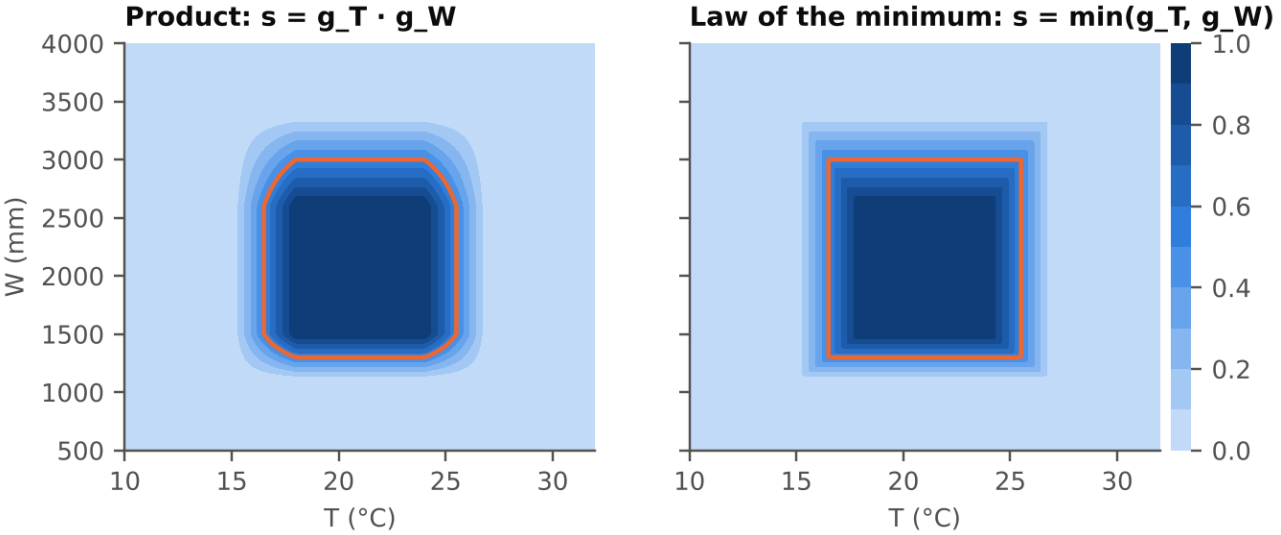


Figure 7. Coffee's suitability over temperature and water under the two rules; the orange line is *s* = 0.5.

4.2 Where crops would grow on the Earth map

Crop	Plots holding it today	Mean s on those plots	Mean s on all land	Land with s ≥ 0.5
Wheat	16	0.18	0.126	12 %
Rice	8	0.23	0.124	13 %
Maize	9	0.68	0.194	18 %
Potato	10	0.23	0.138	14 %
Cotton	4	0.66	0.135	11 %
Coffee	16	0.21	0.069	7 %
Olives	7	0.31	0.07	7 %
Sugar	12	0.33	0.107	12 %

The model was not fitted to the map's resource plots, so they are a check. Where the map maker put maize, cotton, coffee, olives and sugar, the model's suitability is three to five times the land average; for wheat, rice and potato it is about 1.5 times. The potatoes of this map sit in the Andes, which the map draws as flat plains and grassland, so the model sees no highland cool there: altitude on maps that draw mountains as flat land is a known limit (section 15).

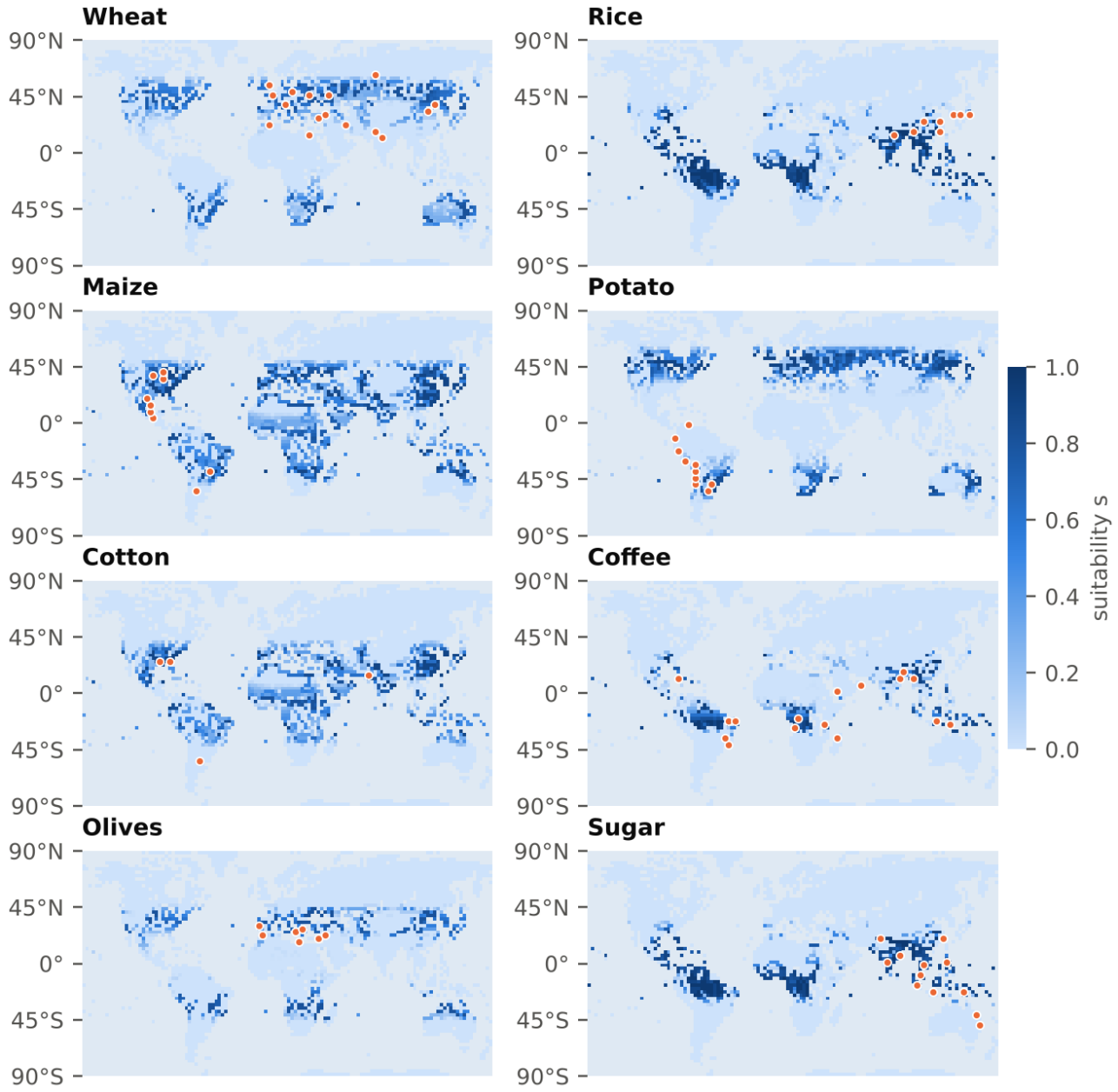


Figure 8. Suitability of eight crops on the Earth map (blue, 0 to 1), with the plots where the map puts the crop today (orange dots). Under this design those dots disappear; the blue is what the land offers.

4.3 What a farm produces

A farm on plot *p* growing crop *c* produces the crop as a good, plus food or commerce as the crop's record says:

$$q_{p,c} = Q_c \, s_{p,c} \, (1 + \tau_c) \, \lambda_p \tag{10}$$

Q_c is the yield of perfect land, τ_c the sum of technology and building bonuses (irrigation, crop rotation, fertilisers), and λ_p the improvement level (a Farm, or a Latifundium stage). The plot's food is part of the same output: grain crops give most of *q* as food, cash crops as commerce and as goods.

Integer yields. The engine keeps plot yields as integers. The DLL would compute in hundredths and hand each worked plot an integer by error diffusion within the city, so the city's total is exact and no plot's yield flickers:

$$y_i = \left\lfloor \sum_{j \le i} \frac{q_j}{100} \right\rfloor - \left\lfloor \sum_{j < i} \frac{q_j}{100} \right\rfloor \tag{11}$$

with the plots taken in a fixed order (the city's plot index). Goods are kept in hundredths in the city's account (section 8) and need no rounding.

Choosing the crop. Options, from simplest: (a) the worker builds "Farm (crop)", one build per crop, valid where $s \geq 0.25$ and the crop is known; (b) one Farm build, and the city governor picks the crop each turn among the known ones by marginal value (section 8.4); (c) as (b), with a switching cost of a few turns of lost output so fields are not replanted every turn. Option (c) is recommended: it keeps workers' actions as they are and gives the AI a single decision rule.

5. Seeds and the spread of crops

A civilization can plant only the crops it knows. Knowledge starts where crops were domesticated and spreads by contact and trade, as wheat, maize and coffee spread on Earth.

5.1 Model

Let $K_{i,c} = 1$ if civilization i knows crop c. At the start, the civilizations near the crop's origin know it. On a WorldBuilder map the origins can be read from where the map puts the crop today: here, a civilization knows the crop if its start lies within 5 plots of one of those plots. Each turn an unknowing civilization learns it with probability

$$h_{i,c} = 1 - \exp\left(-\lambda \sum_j K_{j,c}\, w_{ij}\right), \qquad w_{ij} = e^{-D_{ij}/D_0} \cdot [\,\text{contact}_{ij}\,] \tag{12}$$

D_{ij} is the distance between the two capitals, D_0 the reach of contact, and contact is cut between continents separated by ocean until a technology allows crossing it (here at turn 200; assumption). In a full implementation w_{ij} would be the number of trade routes and open borders between the two, which the game already counts.

Figure 9 shows 2,000 simulated games on the Earth map's 18 starting positions ($\lambda = 0.025$, $D_0 = 18$ plots; assumptions). Wheat starts with 12 civilizations of the Old World and soon reaches all of it; maize starts with 2 (Aztec, Iroquois) and reaches half the civilizations around turn 220, soon after the oceans open; coffee starts with 3 (India, Persia, Arabia).

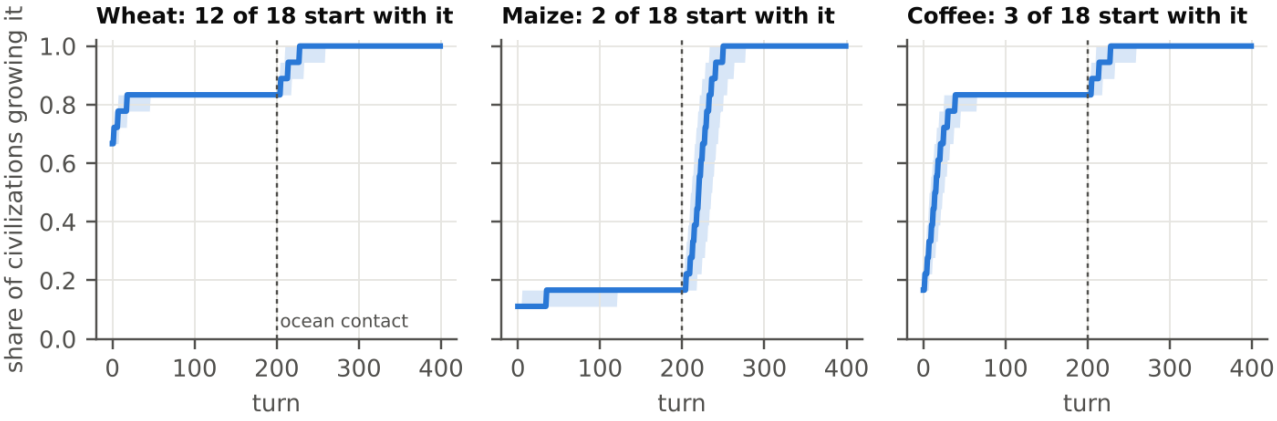


Figure 9. Share of the 18 civilizations that know each crop, over 400 turns (line: median of 2,000 simulated games; band: 10th to 90th percentile). The dashed line is the opening of ocean contact.

5.2 Ways to give seeds

Option	How a civilization gets a crop	Comment
Contact	By the hazard above, from trading partners and neighbours	Organic; needs no new interface
Trade item	Seeds offered on the trade screen like a technology	Needs a new trade item type in the DLL; the screen is the executable's, so it would appear as a technology-like entry (open question)
Expedition	A unit (botanist, explorer) carries seeds home from a foreign region	Gameplay for scouts and colonial powers
Technology	Botany or Biology unlocks crops the civilization has seen	Simple; fits the tech tree

6. Minerals: provinces, grades and depth

Ores are not uniform qualities of land: they come in deposits. In this design a deposit is a province, a field of ore grade that is highest at its centre and fades over a few plots, with reserves in layers that only deeper mines reach.

6.1 Provinces

For a mineral *m* with province centres *k* at plots $\mathbf{x}_k$, the grade at plot **x** is

$$G_m(\mathbf{x}) = \max_k \ \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_k\|^2}{2\sigma^2}\right) \tag{13}$$

with σ = 3 plots (assumption). The centres can come from three sources: (a) on existing WorldBuilder maps, from today's resource plots, which converts the mod's many scenario maps without redrawing them (figure 10 does this for the Earth map); (b) on random maps, from noise biased to hills and peaks; (c) hidden until prospected, so geology and exploration matter. Provinces of grade 0.5 or more on the Earth map: iron 588 plots from 22 centres, copper 520 plots from 20 centres, coal 489 plots from 20 centres, gold 411 plots from 15 centres.

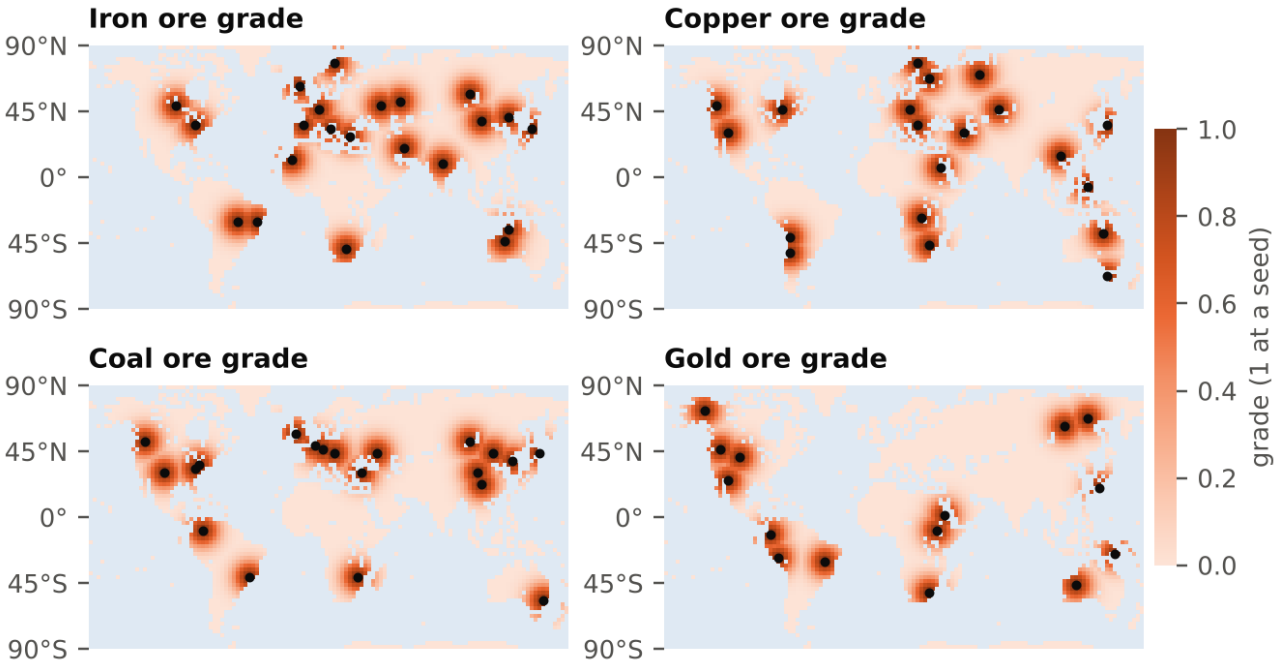


Figure 10. Ore grade of four minerals on the Earth map, with today's resource plots as province centres (black dots).

6.2 Depth and exhaustion

Each province holds reserves in three layers, R_1, R_2, R_3, scaled by the grade. The three mine improvements RoM already has become the three depths: a Mine works layer 1, a Shaft Mine layers 1 and 2, a Modern Mine all three. The extraction rate per turn follows each improvement's production yield in the XML, 1, 2 and 8, times the plot's grade:

$$e_t = \min\left(r_l G,\ \sum_{i \le l} R_{i,t}\right), \qquad R_{i,t+1} = R_{i,t} - e_{i,t} \tag{14}$$

with *l* the mine's depth and the extraction taken from the top layer down. When the layers a mine reaches are empty, it is worked out: the player deepens it (if the technology is known) or abandons it. Figure 11 follows one deposit (layers 120, 400 and 1600 units; assumptions) with the technologies at turns 220 (Explosives) and 340 (Seismology; illustrative). A mine left as it is yields 120 units and is dry at turn 120; deepened when it runs dry it yields 1160; deepened as soon as each technology arrives it yields 1640.

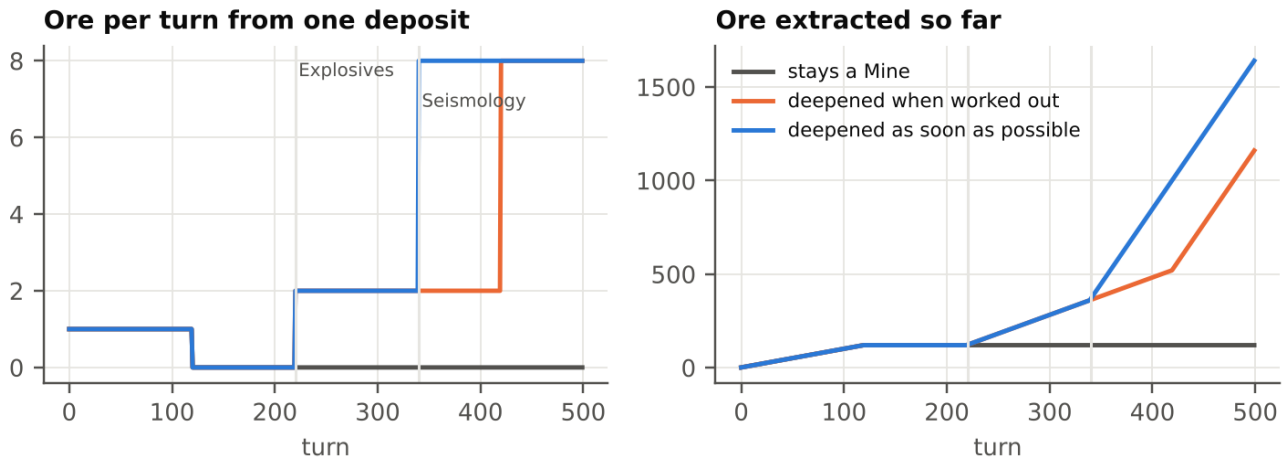


Figure 11. One deposit worked three ways: left as a Mine (gray), deepened when worked out (orange), deepened as soon as possible (blue). Vertical lines: the technologies of the Shaft Mine and the Modern Mine.

Depletion makes geography move over a game: early iron provinces run out, later ones are reached by depth. A further option is the Hotelling rule for the AI: extract more slowly when the good's price is expected to rise faster than the interest of holding gold, which in game terms means saving a nearly worked-out deposit for war.

7. The sea

Instead of fish, crabs and whales on single plots, every sea plot has a carrying capacity K for fish: highest on the continental shelf (coast terrain), at river mouths, and in cold water, lowest in open ocean and under ice (weights are assumptions):

$$K = (0.25 + 0.55\,[\text{shelf}] + 0.2\,[\text{river mouth}])\left(0.55 + 0.45\,\text{clamp}\left(\frac{|\varphi| - 20}{40}\right)\right) \tag{15}$$

On the Earth map the shelf has 2111 plots of 5355 water plots, with mean capacity 0.53 against 0.20 in the open sea.

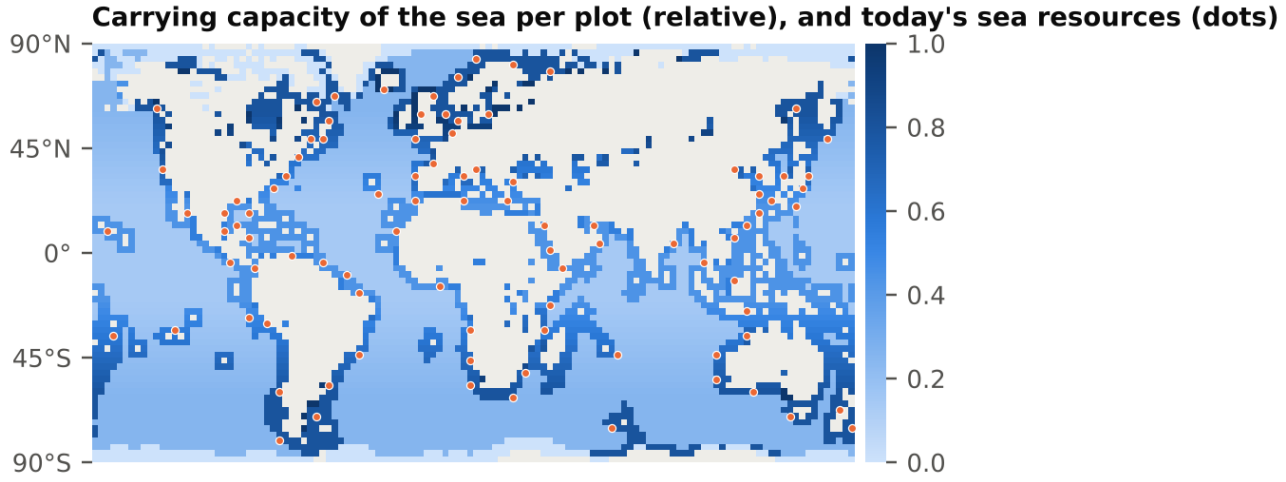


Figure 12. Carrying capacity of the sea on the Earth map, and today's 94 sea resource plots (orange) for comparison.

The stock N of a fishing ground (the sea plots a city works, or a named sea region) grows logistically and is reduced by the catch H:

$$N_{t+1} = N_t + rN_t\left(1 - \frac{N_t}{K}\right) - H_t, \qquad H_{MSY} = \frac{rK}{4} \;\text{ at }\; N = \frac{K}{2} \tag{16}$$

The most that can be taken every turn for ever is the maximum sustainable yield $rK/4$. With r = 0.3 and K = 100 (assumptions) it is 7.5. A catch of 5 keeps the stock near 79 and lands 600 over 120 turns; a catch of 7.5 lands 900; a catch of 9 collapses the stock and lands only 308. Fishing fleets, overfishing, treaties over fishing grounds and naval blockades of them all follow from these two lines.

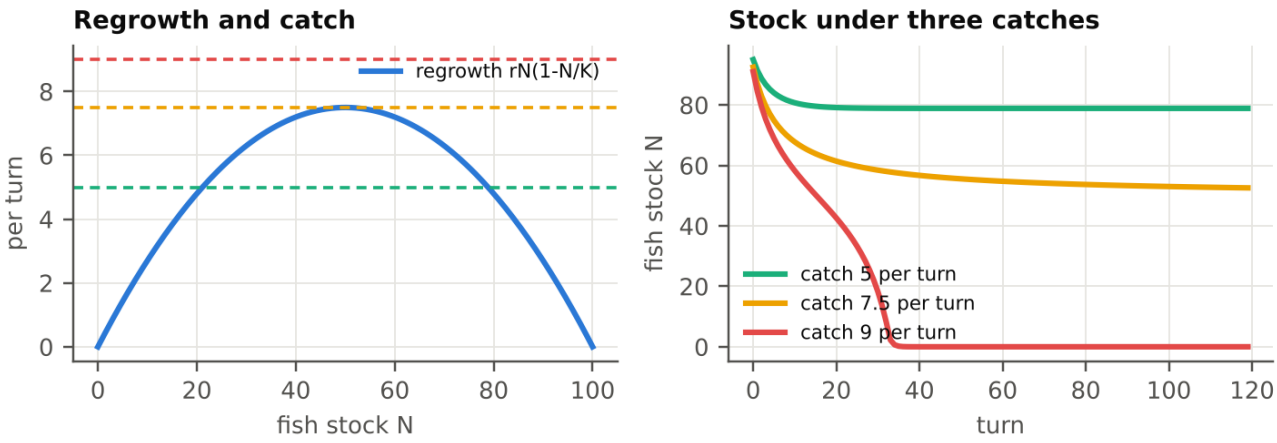


Figure 13. Left: regrowth of the stock against three catch levels; the stock is stable where a catch line meets the curve. Right: the stock over 120 turns under each catch.

8. Goods and quantities

8.1 The city's account

Each city keeps, for every good *g*, what it made, received, used and stored this turn. Within a trade network, surpluses flow to deficits (the network is the engine's plot group, which already carries resources between connected cities):

$$X_{g,\,t+1} = \min\left(C_g,\ X_{g,\,t} + q_{g,\,t} + m_{g,\,t} - u_{g,\,t} - x_{g,\,t}\right) \tag{17}$$

X: stock, *q*: production, *m*: imports, *u*: use by buildings, units and citizens, *x*: exports, *C*: storage capacity (granaries, warehouses, arsenals). Goods that cannot be stored (fresh fish, electricity) have *C* = 0.

8.2 Recipes

A building *k* is a recipe: per run it takes inputs $\mathbf{a}_k$ and gives outputs $\mathbf{b}_k$, up to a capacity of c_k runs per turn. With fixed proportions (Leontief) the runs possible are limited by the scarcest input:

$$n_k = \min\left(c_k,\ \min_g \frac{\alpha_{g,k} X_g}{a_{g,k}}\right), \qquad \alpha_{g,k} = \min\left(1, \frac{a_{g,k} c_k}{\sum_j a_{g,j} c_j}\right) \tag{18}$$

$\alpha_{g,k}$ is building *k*'s share of good *g* when several buildings want it: proportional rationing by demand. Recipes are evaluated in the order of their dependency graph (ore before steel before arms), once per network per turn. A production function with substitution (constant elasticity, CES) is the alternative where inputs can replace each other, such as coal and charcoal in iron making:

$$n_k = c_k \left(\sum_g \theta_g \left(\frac{X_g}{a_{g,k} c_k}\right)^{\rho}\right)^{1/\rho}, \qquad \rho < 1 \tag{19}$$

Figure 14 shows one network for one turn: 6/3/2/2 per turn of iron ore, coal, sulphur and copper, a steel mill (2 ore and 1 coal give 2 steel), an arms factory (1 steel and 1 sulphur give 1 arms) and a cartridge works (1 copper and 1 sulphur give 2 ammunition). The steel mill runs at full capacity (3 runs); sulphur is shared by the two users, so the arms factory runs 1.2 times and the cartridge works 0.8 times, and 4.8 steel is left over.

8.3 What a good is worth

The value of one more unit of a good is the change in what the network produces with it, valued at the AI's weights for food, production, commerce and goods:

$$v_g = \frac{\partial V}{\partial q_g} \approx V(\mathbf{q} + \mathbf{e}_g) - V(\mathbf{q}) \tag{20}$$

In the example one more sulphur is worth **+2.08**, ten times one more iron ore (**+0.20**), because sulphur binds both factories and iron binds nothing. No fixed weight per good can say this; the AI must compute it. This is the same principle the diplomacy change of merge request !63 applies to today's resources: value a good by what it changes in the valuer's own cities.

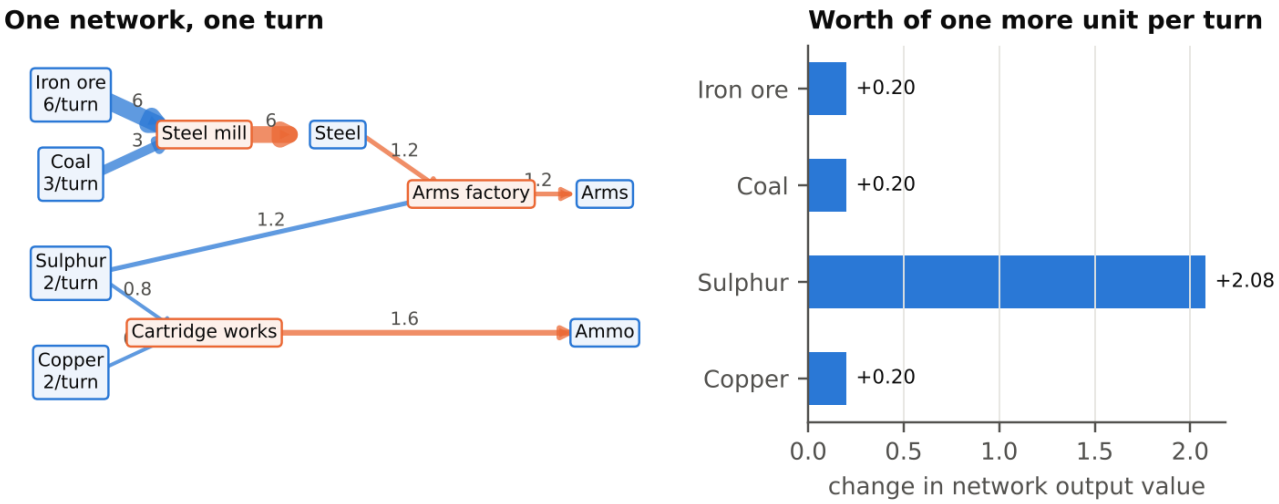


Figure 14. Left: goods flows in one network for one turn (arrow width: quantity). Right: the value of one more unit per turn of each raw good, by recomputing the network.

8.4 Luxuries and health per head

Today a luxury gives a fixed bonus to every city. With quantities, a city's happiness from a good grows with consumption per head, with diminishing returns, so a large city needs more of it:

$$\eta_g = H_g(1 - e^{-c_g/k_g}), \qquad c_g = \frac{10\,u_g}{\text{population}} \tag{21}$$

H_g is the most the good can give (2 here), k_g the consumption per ten citizens that gives 63 % of it. The same form serves health from food variety. Figure 15 compares it with today's +1.

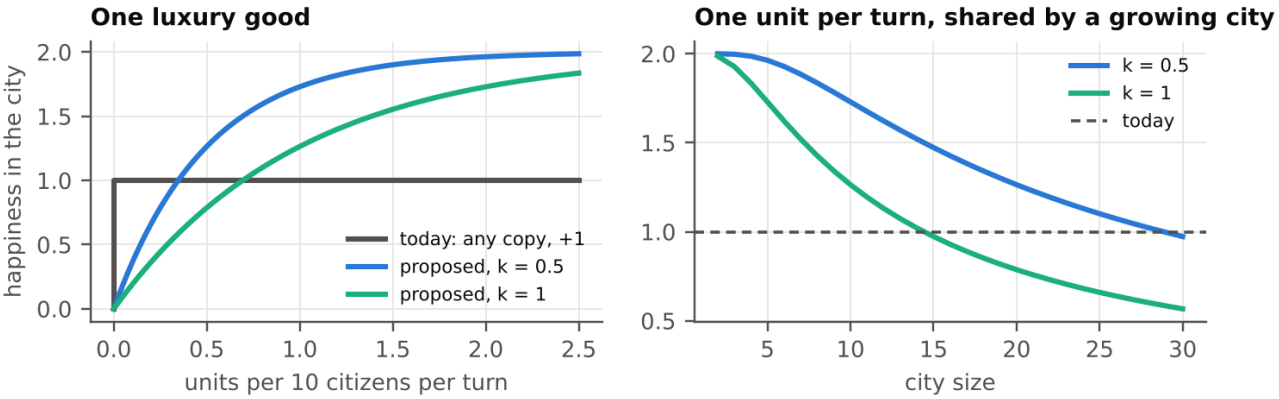


Figure 15. Left: happiness from one luxury as consumption per ten citizens grows, today and under two values of *k*. Right: one unit per turn shared by a city as it grows.

8.5 Into the engine

Building and unit requirements, the trade screen and happiness rules read the city's resource counts. The bridge sets each count from the account: a city that has, per turn, at least one unit of good *g* available (made, received or drawn from store) holds *g* once for every unit:

$$\text{FreeBonus}_g = \left\lfloor \frac{X_g + q_g + m_g}{U_g} \right\rfloor \tag{22}$$

where U_g is the good's unit size. The counts are updated through `changeFreeBonus` once per turn, after the network's recipes have run. With happiness per head (8.4) the engine's own luxury rule is switched off for goods and the new rule applied.

9. Industry and war

Arms become goods that are made, stored and used. The ladder of options, from today:

Level	Rule	What changes in play
0 (today)	A unit needs a resource present	Having one copy is enough for an empire
1	Building a unit costs goods: a Rifleman 2 arms, a Cannon 3 steel and 2 arms (illustrative)	Armies need industry; output caps how fast an army grows
2	Units in the field use supplies each turn (ammunition, fuel)	Long wars drain industry; peace lets stores refill
3	Battles use ammunition; a unit without it fights at reduced strength	Blockades and raids on factories decide wars

The arsenal's stock follows the account of section 8.1 with the demand of war added. When the stock is empty, demand is met only from current production:

$$\text{met}_t = \frac{\min(D_t,\ X_t + q_t)}{D_t} \tag{23}$$

Figure 16 runs a 90-turn war (turns 100 to 190) on an arsenal of capacity 60: the store is empty at turn 111, new factories raise production at turn 125, a blockade cuts it at turns 150 to 175, and the share of demand met falls to 38 %.

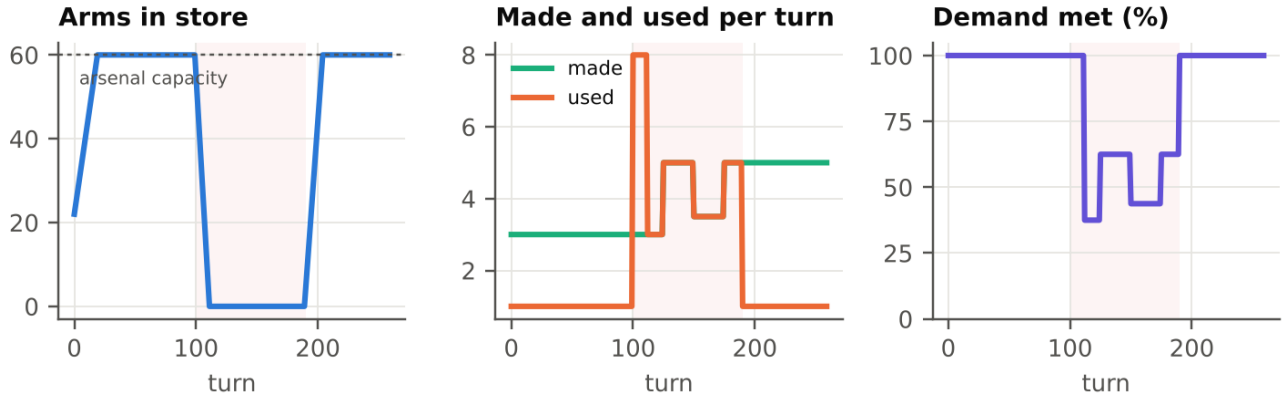


Figure 16. An arsenal through a war (shaded): stock, production and use, and the share of the army's demand met.

10. Trade and prices

Goods move between civilizations through the trade screen (as resources do today) and along trade routes. With quantities, a price per good lets the AI judge deals and lets a scarce good earn its holder something:

$$p_g = p_g^0 \,\mathrm{clamp}\left(\left(\frac{D_g}{S_g}\right)^{1/\varepsilon},\ 0.2,\ 5\right) \tag{24}$$

D and S are the world's (or the trading partners') demand and supply of the good, ε its elasticity, p^0 its base price. A monopolist that sells s units against demand D earns $p(s)\,s$; with $\varepsilon = 1$ that is constant in s wherever the price cap does not bind, so selling more earns nothing more, and with $\varepsilon < 1$ selling less earns more: the incentive to hoard a monopoly good, which coffee or spices had historically.

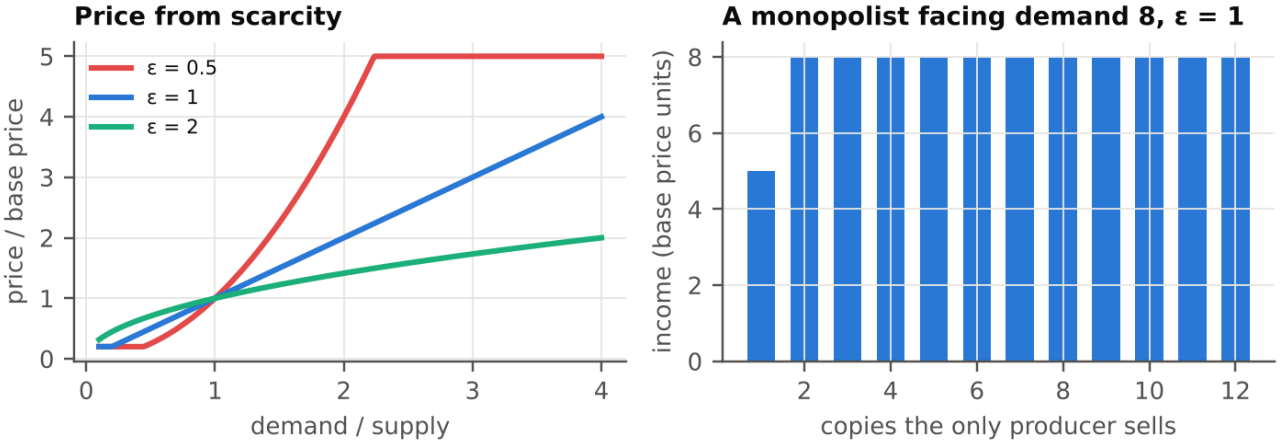


Figure 17. Left: price against the ratio of demand to supply for three elasticities. Right: a monopolist's income as it sells more copies.

For the AI, a deal's worth is the sum of marginal values (section 8.3) of what it gets minus what it gives, and gold is worth what it buys at current prices. Where the game is not a market (most of history), prices can be used only inside the AI's valuation and never shown.

11. How the mechanics interlock

The mechanics form loops. Good land feeds people; people work mines; ore becomes steel and arms; arms win land. Farming wears the soil, which lowers the land's quality until fertilisers or rest restore it; seeds and trade raise what the land can grow.

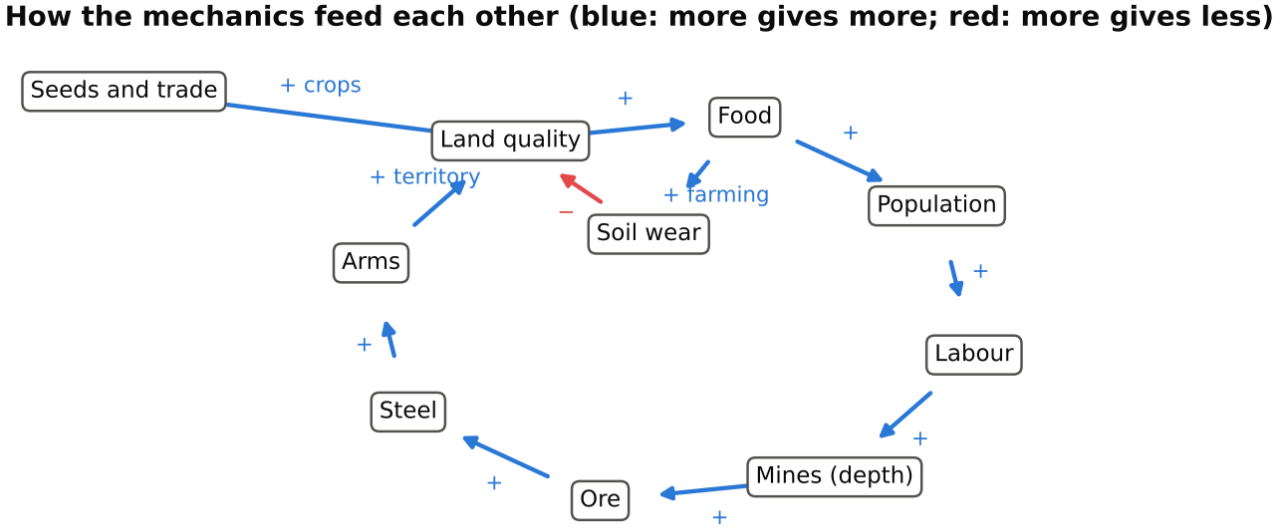


Figure 18. Feedback between the mechanics. Each arrow says how a change in one quantity moves the next.

Figure 19 runs a deliberately small version of the loop for 300 turns: a realm with the 60 best wheat plots of the Earth map (mean suitability 0.81), six iron deposits with the layers of section 6, soil that wears until fertilisers arrive at turn 200, shaft mines at 160 and modern mines at 240, and a war at turns 180 to 220. Population stalls as the soil wears and grows again with fertilisers; the war empties the arsenal (lowest store 0) because the mines are still shallow; modern mines then raise ore and steel several times over. The point is not the numbers, which come from illustrative parameters, but that the outcome depends on the order in which land, technology and war arrive.

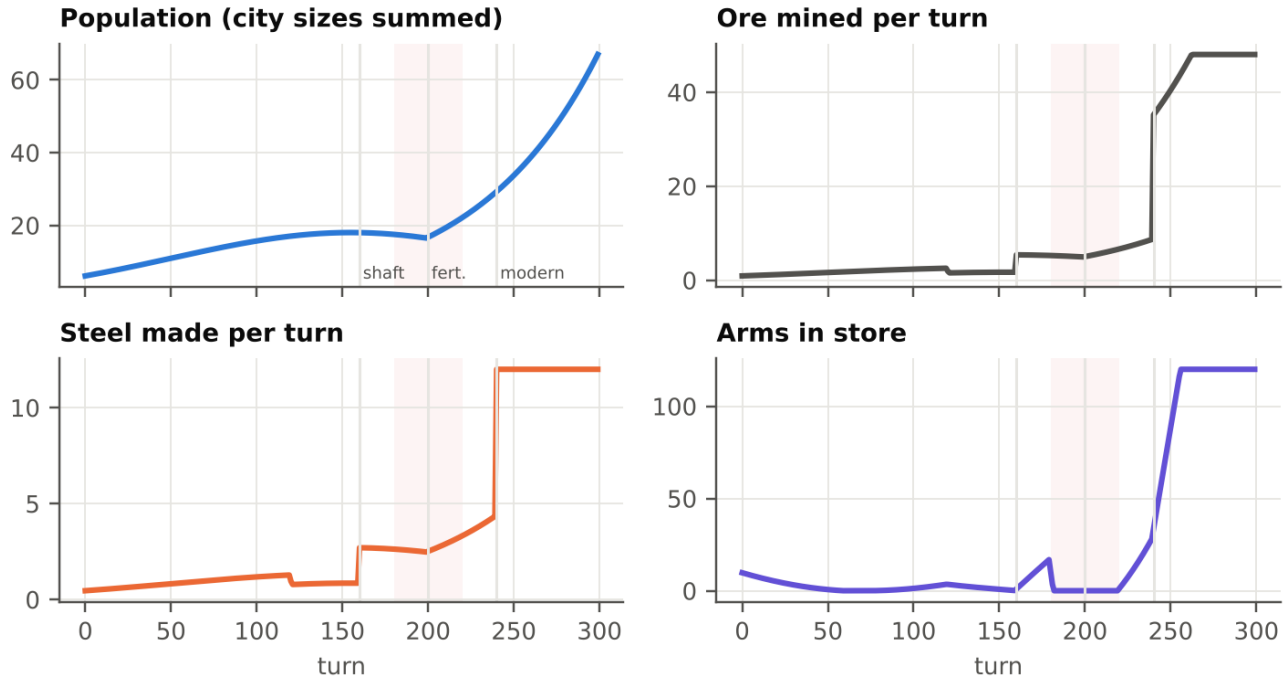


Figure 19. One realm over 300 turns with the loop of figure 18 (war shaded; vertical lines: shaft mines, fertilisers, modern mines).

12. Fitting it into the engine

Civilization IV splits into the executable (rendering, interface screens, map display), the DLL (all rules; source available), XML (data) and Python (screens, map scripts, events). The proposal changes the DLL, XML and Python; it works around the executable.

Piece	Layer	What changes	Risk
Plot qualities	DLL	New fields on CvPlot: temperature, water, season, soil, province grades, reserves, crop, fish stock (about 20 bytes per plot); computed at game start; saved with a version flag	Low
Crop and goods records	XML + DLL	Crop windows and yields on CIV4BonusInfos entries (new tags); recipes on buildings (input and output quantities); unit goods costs	Low
Plot yields	DLL	CvPlot::calculateNatureYield and calculateImprovementYieldChange read qualities instead of the bonus	Medium
City accounts and recipes	DLL	New per-city arrays; a per-network pass each turn in CvPlayer::doTurn	Medium
Bridge to resources	DLL	changeFreeBonus from the accounts (section 8.5)	Low
No tokens on the map	DLL + Python	Map scripts place no resources (CvMapGenerator::addBonuses skipped); WorldBuilder maps converted at load (provinces and crop origins from their resource plots, then the plots cleared)	Medium
Showing the land	Python	Overlays of suitability, grade and fish through colored plots (the engine API the strategy overlay uses); plot help text; a land screen	Low
Resource models on the map	Executable	Not drawn any more, as no plot holds a resource; improvements (farms, mines, boats) still drawn	None
Trade screen	Executable	Unchanged: goods appear as resources do now	Low
AI	DLL	Marginal values of goods; crop choice; mine deepening; worker plans; trade and war plans with stocks	High
Saves	DLL	Old saves load with qualities computed at load; new saves are not readable by old DLLs	Medium

Cost per turn. Suitability is static: 8432 plots times 8 crops is 67456 evaluations once at game start. Per turn the new work is one pass over each network's recipes (tens of buildings) and one over its cities' accounts (cities times goods), well below what the AI already spends on city governors each turn.

Determinism. Everything above is deterministic except seed spreading, which must use the game's synchronised random number generator (as all game rules do) so multiplayer and the headless test harness stay in sync.

13. Three ways to build it

	A. Bridge	B. Quantities	C. Full economy
Land qualities	Climate, soil, provinces, fish capacity	Same	Same, plus soil wear, depletion, fish stocks
Crops	Farms grow known crops where suitable	Same, with yields by suitability	Same, with seed spreading and switching costs
Goods	Present or absent per city (resource counts)	Counted per turn; recipes use inputs	Stored, traded with prices
Luxuries	As today	Per head (8.4)	Per head
Military	Gates as today	Unit build costs in goods	Supplies and ammunition in war
AI work	Crop choice; valuation of land	Marginal values; production planning	Stocks, prices, war logistics
Size	Smallest; playable alone	Medium	Largest
Risk to the game	Low	Medium: AI must plan production	High: AI must plan stocks and trade

Each path contains the previous one. **A** already removes the tokens and gives land its qualities; **B** makes quantities matter; **C** adds time (stores, depletion, regrowth) and markets.

14. Roadmap and how each step is judged

Under the repository's rules each step is a registered prediction, built, then judged: a safety claim (no crash, not slower) by 5 seeded harness runs per arm, a claim that play gets better by at least 30, and what only a human game can show by the user's games.

Step	Content	Judged by
1	Plot qualities computed and shown (overlays, help text); nothing else changes	Safety runs; the overlays in the game
2	Tokens removed; farms grow crops by suitability; mines and boats by provinces and sea; bridge to resource counts (path A)	Safety runs; play runs on cities founded and food per city against the current game
3	AI for crop choice and land valuation	Play runs: food and production per city of AI civilizations
4	Counted goods and recipes; luxuries per head (path B)	Safety runs; play runs; the user's games for balance
5	Unit costs in goods; mine depth and exhaustion	Play runs on war outcomes against fixed opponents
6	Stocks, seeds spreading, fish stocks, prices (path C)	Play runs; the user's games

15. Assumptions and open questions

Assumptions (design parameters to tune, not facts):

- Temperature profile 27 - 52 $(|\phi|/90)^2$ °C, lapse 3 °C for hills and 12 °C for peaks, seasonal amplitude formula.
- Rain belts and inland decay; terrain rain values; blend weight 0.7 for the drawn terrain; river irrigation 700 mm.
- Soil values per terrain; soil exponent $\beta = 0.5$; the crop windows of table A1.
- Seed spreading: $\lambda = 0.025$, reach 18 plots, ocean contact at turn 200, origins within 5 plots of today's resource plots.
- Province width $\sigma = 3$ plots; reserve layers 120, 400 and 1,600 units; technology turns 220 and 340.
- Fish regrowth $r = 0.3$, capacity weights; price clamp 0.2 to 5; recipe quantities and unit costs in sections 8 and 9.

Open questions

- Maps that draw mountains as flat land give the model no altitude (the Andes potatoes of section 4.2). A map field for altitude, or peaks and hills in the maps, would fix it.
- Whether the trade screen can show seeds as an item: the screen is the executable's; resources and technologies are the item types it knows.
- How the player reads the land without tokens: overlays and help text are proposed; whether that is enough is for play to show.
- Balance across eras: how much land quality should matter against buildings and technology.
- How far the AI can be taught production planning (path B) before path C is worth starting.

Appendix: parameters and notation

Table A1. Crop windows used in the figures (assumptions; would be XML fields).

Crop	Temperature window a, b, c, d (°C)	Water window (mm)	Season (months)	Coldest month at least (°C)
Wheat	3, 8, 16, 23	250, 450, 900, 1500	4	-
Rice	16, 21, 29, 34	900, 1500, 3000, 4500	6	5
Maize	10, 15, 26, 31	450, 650, 1300, 2000	5	-
Potato	2, 7, 15, 21	400, 600, 1200, 1900	3	-
Cotton	15, 20, 28, 32	450, 700, 1300, 2000	7	0
Coffee	15, 18, 24, 27	1100, 1500, 2600, 3400	12	10
Olives	11, 14, 20, 24	250, 400, 800, 1200	8	0
Sugar	19, 23, 30, 34	1000, 1400, 2600, 3800	11	8

Notation

Symbol	Meaning
ϕ	latitude, degrees (negative south)
T, T_m, T_{min}	annual mean, monthly and coldest-month temperature
A	seasonal amplitude
L	growing season, months
d	distance to water, plots
P, W	rain; water available to crops
S	soil fertility, 0 to 1
s_c	suitability of a plot for crop c
G_m	ore grade of mineral m
R_i	reserves of layer i
K, N, r	fish capacity, stock, regrowth rate
X, q, u, m, x, C	stock, production, use, imports, exports, storage capacity of a good
n_k, c_k	runs and capacity of recipe k
v_g	marginal value of good g
p_g, ε	price and elasticity of good g

Sources. GitLab aretemaxxing-group/rise-of-mankind `main 3bec049`: Assets/XML/Terrain (CIV4TerrainInfos, CIV4FeatureInfos, CIV4BonusInfos, CIV4ImprovementInfos), Assets/XML/Buildings/CIV4BuildingInfos.xml, Assets/XML/Units (CIV4UnitInfos, CIV4BuildInfos), Assets/XML/GameInfo/CIV4WorldInfo.xml, PrivateMaps/Earth 18 preset civs, Huge.CivBeyondSwordWBSave, and the DLL source (CvPlot.cpp, CvCity.cpp, CvGame.cpp). Scripts and data: GertaBool/RoM research/economy.